

Project Title

Design and Implementation of a Tiered Token Bucket Rate Limiter for DDoS Mitigation in Microservice Architectures

1. Introduction

Distributed Denial of Service (DDoS) attacks remain a critical threat to modern web infrastructure. By overwhelming a target server with a flood of illegitimate traffic, attackers can render services unavailable to genuine users, causing significant financial and reputational damage. In microservice architectures, a single unprotected endpoint can become a bottleneck that cascades failure throughout the entire system. This project aims to design and implement a robust Application-Layer (Layer 7) Rate Limiter using the Token Bucket Algorithm. The system acts as a shield, intercepting incoming traffic and enforcing strict usage quotas to ensure service stability.

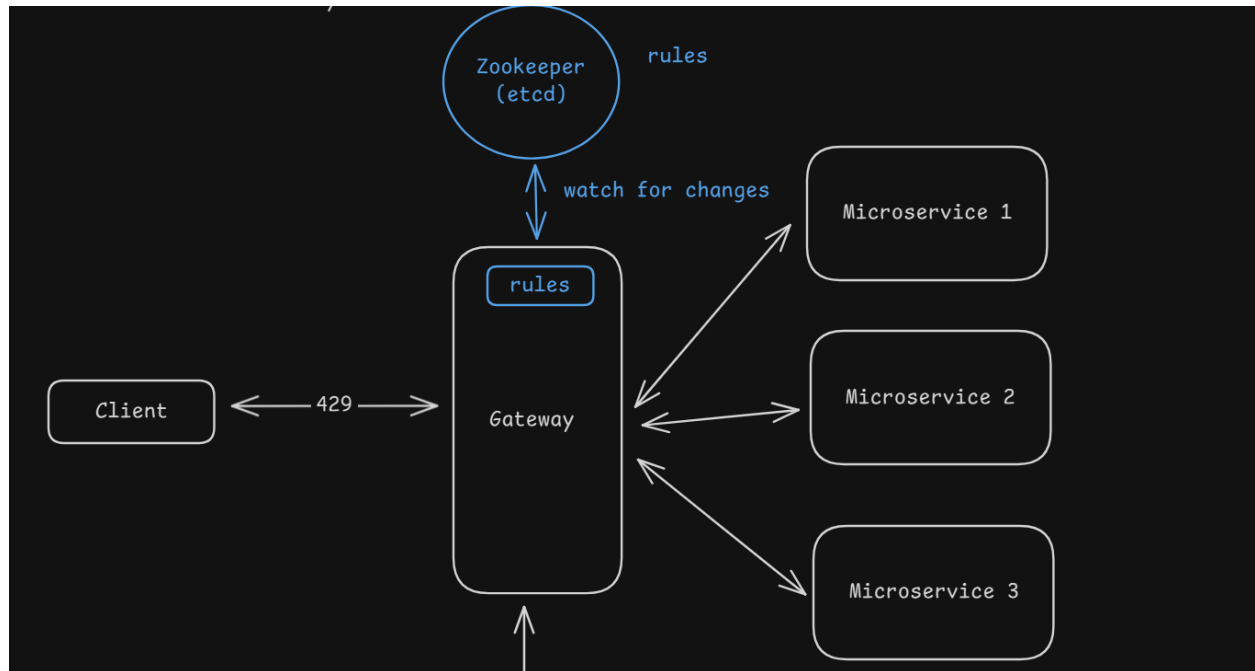
2. Cybersecurity Context: The "Availability" Pillar

This project directly addresses the Availability component of the CIA Triad (Confidentiality, Integrity, Availability), a foundational model in information security.

- **DDoS Mitigation:** The primary goal is to prevent resource exhaustion attacks where an adversary attempts to deplete the server's CPU, memory, or connection pool.
- **Traffic Shaping:** By regulating the flow of requests, the system ensures fair usage among clients, preventing "noisy neighbors" from degrading performance for others.
- **Attack Surface Reduction:** Implementing the Rate Limiter at the API Gateway level (the entry point) ensures that malicious traffic is rejected before it ever reaches the expensive backend microservices, reducing the overall attack surface.

3. System Architecture

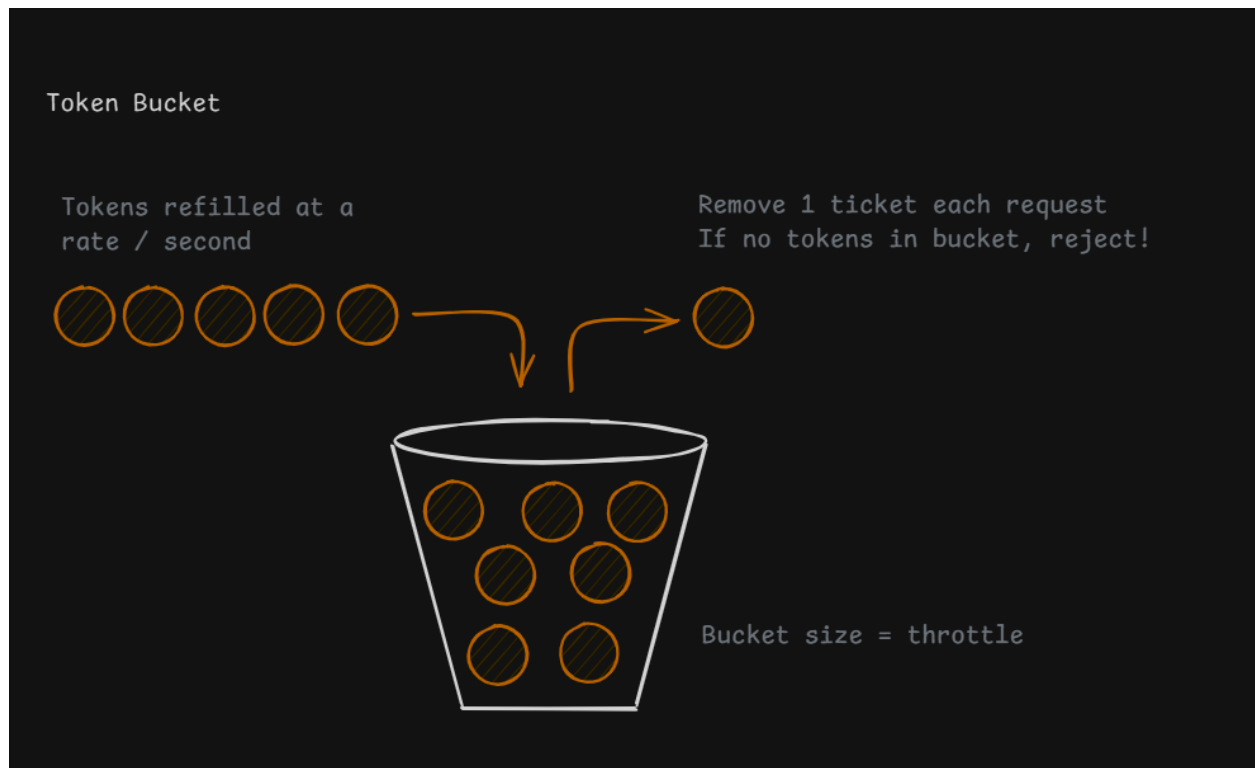
The system is architected as a centralized **API Gateway** acting as a reverse proxy.



This diagram illustrates the high-level architecture of the proposed system. All external client traffic is routed through a central **API Gateway**, which acts as a reverse proxy and security enforcement point. Gateway is responsible for enforcing rate-limiting **rules**, rejecting excess traffic with an **HTTP 429 (Too Many Requests)** response, and forwarding legitimate requests to the appropriate backend **Microservices (1, 2, and 3)**. A configuration management component (e.g., **Zookeeper/etcd**) holds the dynamic rate-limiting rules, which the Gateway watches for real-time updates. This design ensures that the backend microservices are protected from traffic spikes and Denial-of-Service attacks.

4. Algorithmic Design: The Token Bucket

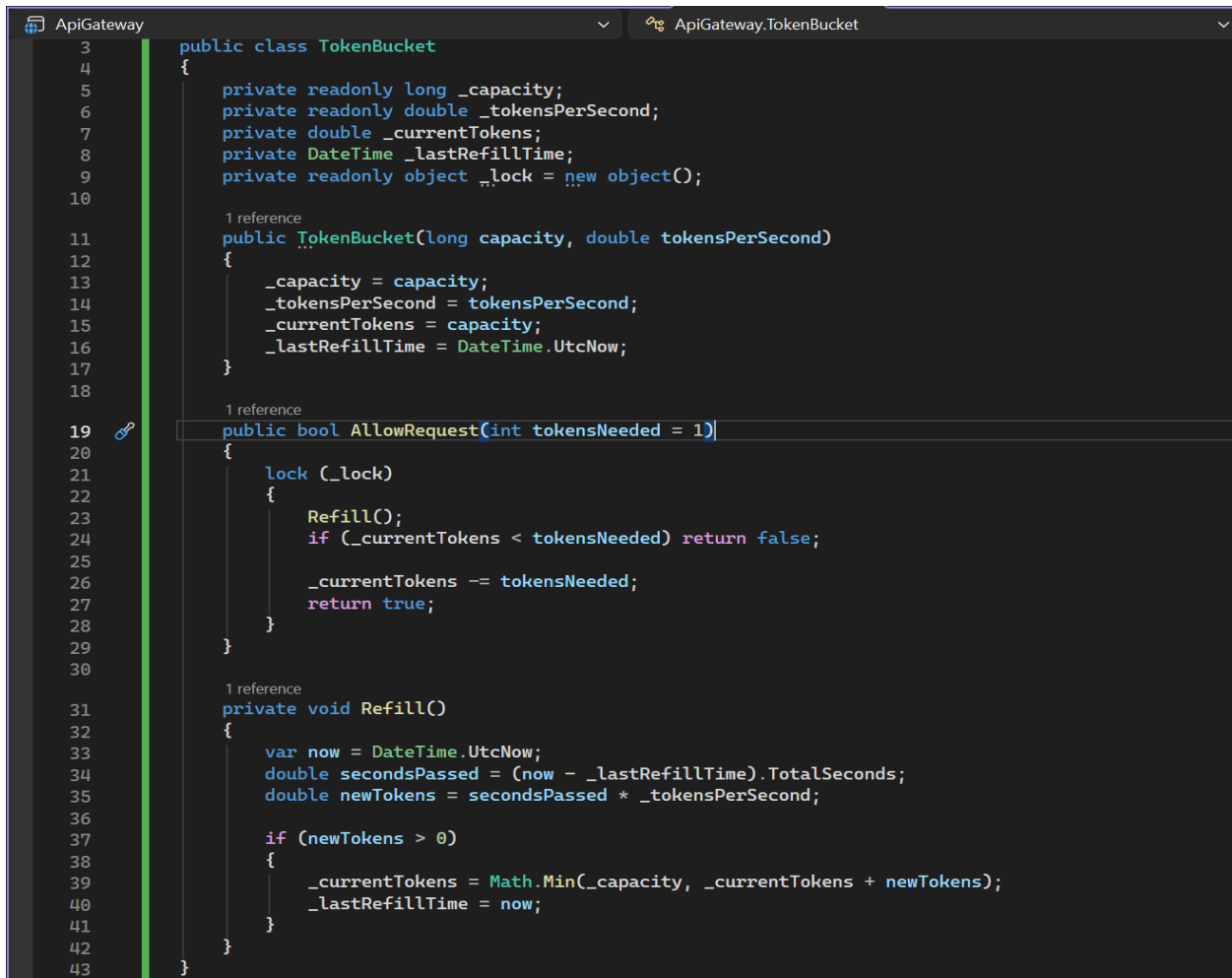
To provide granular control over traffic bursts, we utilized the **Token Bucket Algorithm**. Unlike simple fixed-window counters, the Token Bucket allows for short bursts of traffic while enforcing a strict long-term average.



The rate-limiting logic is implemented using the **Token Bucket algorithm**, as illustrated in the figure above. In this model, every unique user (identified by IP address or API Key) is assigned a virtual "bucket" that holds a specific number of tokens. The system operates on two core rules: first, the **Request Cost**, where the system attempts to remove one token from the user's bucket for each incoming request. If the bucket has sufficient tokens, the request is allowed; however, if the bucket is empty, the request is rejected immediately with an **HTTP 429 (Too Many Requests)** status. Second, the **Refill Mechanism** ensures sustained availability by adding tokens back into the bucket at a fixed refill rate (e.g., 1 token per second) until the bucket reaches its maximum capacity. This design effectively allows users to handle short "bursts" of traffic up to the bucket size while strictly enforcing a limit on the long-term request rate.

5. Technical Implementation

The solution was implemented in **C# (ASP.NET Core)** for the Gateway and **Python** for the attack simulation.



```
3 public class TokenBucket
4 {
5     private readonly long _capacity;
6     private readonly double _tokensPerSecond;
7     private double _currentTokens;
8     private DateTime _lastRefillTime;
9     private readonly object _lock = new object();
10
11     1 reference
12     public TokenBucket(long capacity, double tokensPerSecond)
13     {
14         _capacity = capacity;
15         _tokensPerSecond = tokensPerSecond;
16         _currentTokens = capacity;
17         _lastRefillTime = DateTime.UtcNow;
18     }
19
20     1 reference
21     public bool AllowRequest(int tokensNeeded = 1)
22     {
23         lock (_lock)
24         {
25             Refill();
26             if (_currentTokens < tokensNeeded) return false;
27             _currentTokens -= tokensNeeded;
28             return true;
29         }
30
31     1 reference
32     private void Refill()
33     {
34         var now = DateTime.UtcNow;
35         double secondsPassed = (now - _lastRefillTime).TotalSeconds;
36         double newTokens = secondsPassed * _tokensPerSecond;
37
38         if (newTokens > 0)
39         {
40             _currentTokens = Math.Min(_capacity, _currentTokens + newTokens);
41             _lastRefillTime = now;
42         }
43     }
```

Figure 3: C# Implementation of the Token Bucket Class

Figure 3 presents the core C# code for the Token Bucket class, highlighting the thread-safety mechanisms essential for handling concurrent attacks. The Allow Request method utilizes a lock primitive to ensure that token consumption and refill calculations are atomic, preventing race conditions when multiple threads access the bucket simultaneously. The code also demonstrates the 'lazy refill' strategy, where the token count is updated based on the elapsed time (DateTime.UtcNow) only when a new request is received, ensuring O(1) computational efficiency.

6. Experimental Results

To validate the system, we conducted a series of simulated DDoS attacks using a multi-threaded Python script. The dashboard below shows the system's baseline state before the attack.

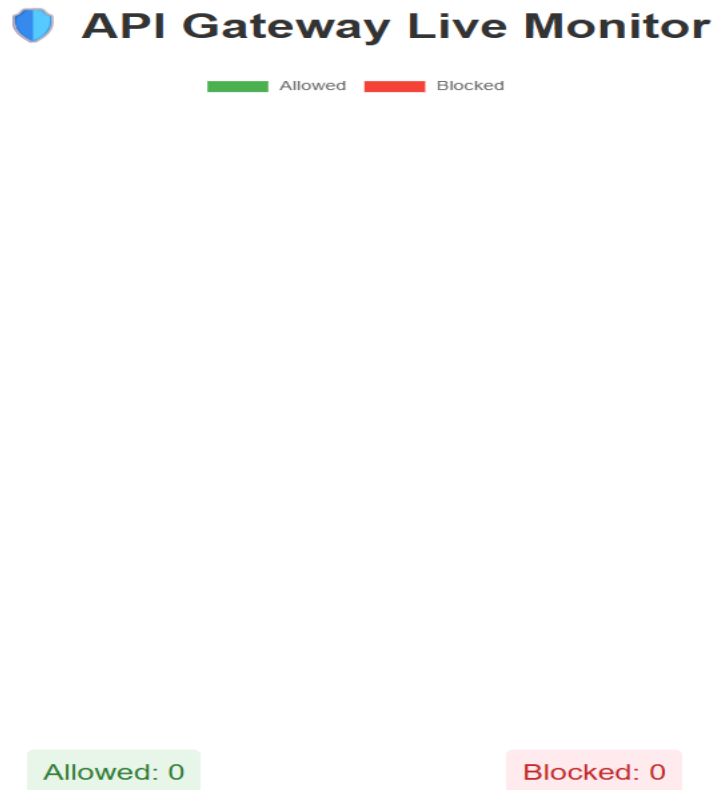


Figure 4: Gateway Dashboard - Initial Idle State

This image in Figure 4 displays the API Gateway Live Monitor in its initialized state prior to the commencement of load testing. The dashboard confirms that the system is online, with both 'Allowed' and 'Blocked' request counters reset to zero. This baseline state validates that no phantom traffic is being recorded and that the monitoring middleware is correctly ignoring its own polling requests.

Scenario A: Free Tier Attack (Baseline Protection) We simulated a burst of 100 concurrent requests from a standard, unauthenticated user. The policy was set to a capacity of **10 tokens**.

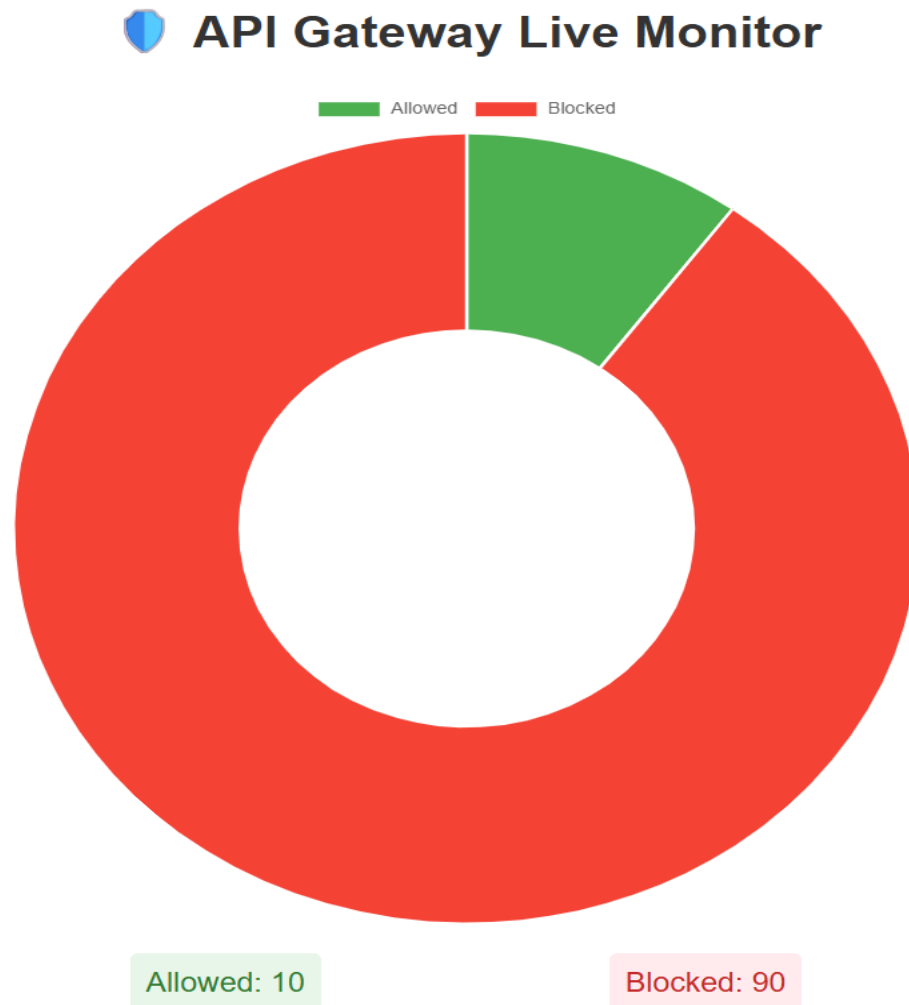


Figure 5: Free Tier Load Test Results (100 Request Burst)

These figures depict the system's response to a burst attack under the default 'Free Tier' policy (Capacity = 10). The Python attacker script sent 100 concurrent requests in under 1 second. The terminal output confirms that exactly **10 requests were passed** (matching the bucket capacity) and **90 requests were blocked** (HTTP 429). The graphical dashboard corroborates this data, showing a dominant red sector (90%) representing the mitigated traffic, effectively demonstrating the system's ability to protect backend resources from unauthorized volume.

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

- Worker 98: BLOCKED by Rate Limiter
- Worker 99: BLOCKED by Rate Limiter
- Worker 78: BLOCKED by Rate Limiter
- Worker 90: BLOCKED by Rate Limiter

=====
🚩 ATTACK SUMMARY
=====
🕒 Duration:      0.78 seconds
🚀 Total Sent:    100
🟢 PASSED (200):  10  <-- Legitimate Traffic
🔴 BLOCKED (429): 90  <-- Mitigated Attacks
❌ FAILED:        0
=====
PS D:\University\Security\TokenBucket> █
```

Figure 6: Free Tier Load Test Results (Terminal Log)

The terminal output confirms that exactly 10 requests were passed (matching the bucket capacity) and 90 requests were blocked (HTTP 429). This demonstrates the effectiveness of the Rate Limiting algorithm and its implementation.

Scenario B: Gold Tier Attack (Business Logic Integration) To demonstrate flexibility, we simulated a "VIP" client using a specific API Key (GOLD-USER-777). The system dynamically upgraded the bucket capacity to **50 tokens** with a **5x faster refill rate**.

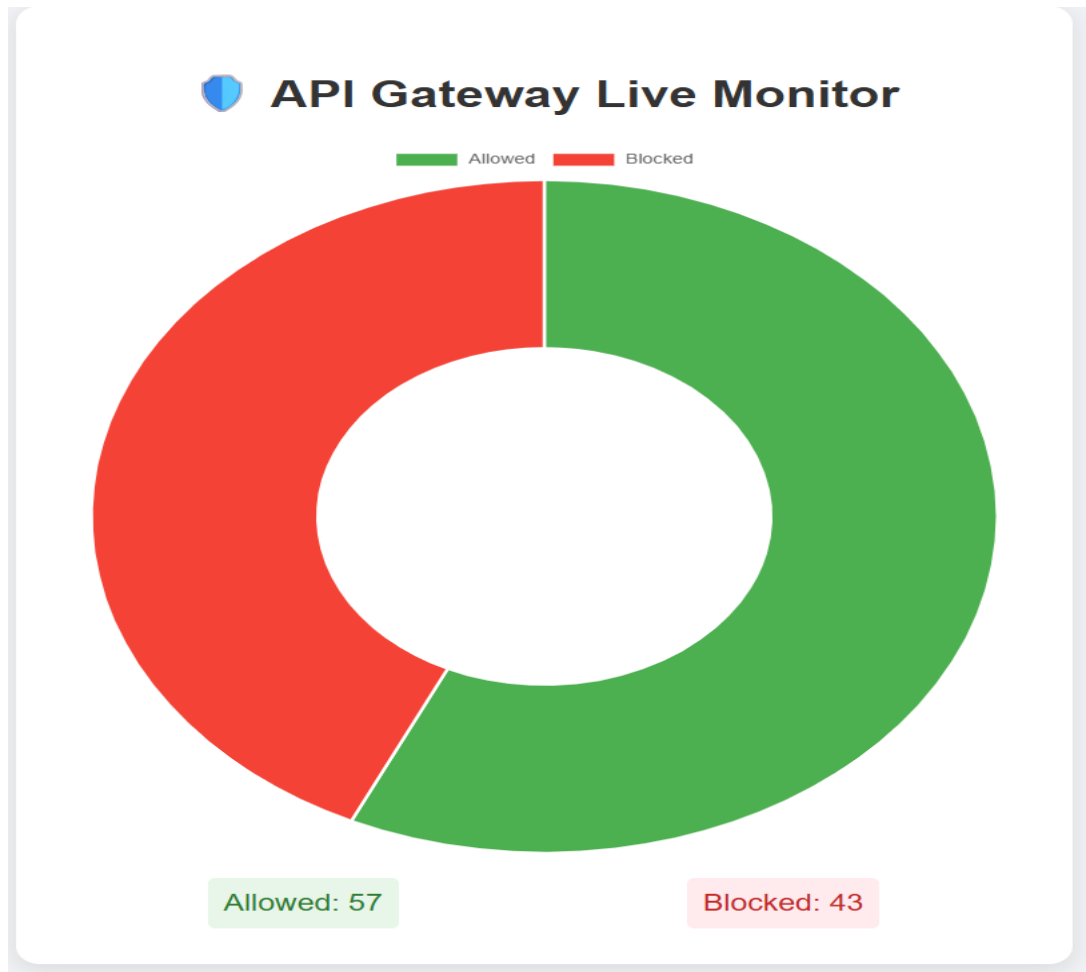


Figure 7: Gold Tier (VIP) Load Test Results

Figure 7 demonstrates the **Tiered Rate Limiting** capability. Upon detecting a valid VIP API Key (GOLD-USER-777), the Gateway dynamically upgraded the user's policy to the Gold Tier (Capacity = 50, Refill = 5/sec). The results show **58 allowed requests** —comprising the initial 50-token burst capacity plus approximately 8 additional tokens that were generated by the high-speed refill rate during the 2.8-second attack window. This validates the system's ability to apply dynamic business rules and support high-performance clients without compromising security architecture.


```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

- Worker 98: BLOCKED by Rate Limiter
- Worker 91: BLOCKED by Rate Limiter
- Worker 94: BLOCKED by Rate Limiter
- Worker 99: BLOCKED by Rate Limiter
✓ Worker 17: Request Allowed (2364ms)
✓ Worker 55: Request Allowed (2181ms)
✓ Worker 44: Request Allowed (2207ms)
✓ Worker 4: Request Allowed (2494ms)
✓ Worker 6: Request Allowed (2643ms)

=====
🏠 ATTACK SUMMARY
=====
🕒 Duration:      2.82 seconds
🚀 Total Sent:    100
🟢 PASSED (200):  58  <-- Legitimate Traffic
🔴 BLOCKED (429): 42  <-- Mitigated Attacks
❌ FAILED:        0

=====
PS D:\University\Security\TokenBucket> |
```

Figure 8: Gold Tier Load Test Results (Terminal Log)

The results show 58 allowed requests—comprising the initial 50-token burst capacity plus approximately 8 additional tokens that were generated by the high-speed refill rate during the 2.8-second attack window. This validates the system's ability to apply dynamic business rules and support high-performance clients without compromising security architecture.

7. Scalability & Future Work

While the current implementation uses an in-memory Concurrent Dictionary for low-latency token storage, this design is stateful to a single Gateway instance. In a production environment with a Load Balancer and multiple Gateway instances, this would lead to a "Split-Brain" issue. To mitigate this in a scaled architecture, future work would involve moving the Token Bucket state to a **Distributed Cache (Redis)** using Lua scripts to ensure atomicity across distributed nodes.

8. Conclusion

The experiment successfully demonstrated that a Middleware-based Token Bucket is an effective, low-latency defense against volume-based DDoS attacks. By implementing tiered policies, the system not only secures the infrastructure but also enables business-aligned traffic management, ensuring that critical services remain available even under heavy load.